

Table 1: The test accuracy (%) on the MNIST addition (**T1**).

Methods	Number of digits per number (N)			
	1	2	3	4
NeurASP	97.3 ± 0.3	93.9 ± 0.7	timeout	timeout
DeepProbLog	97.2 ± 0.5	95.2 ± 1.7	timeout	timeout
DeepStochLog	97.9 ± 0.1	96.4 ± 0.1	94.5 ± 1.1	92.7 ± 0.6

Table 2: The accuracy (%) on the HWF dataset (**T2**).

Method	Expression length			
	1	3	5	7
NGS	90.2 ± 1.6	85.7 ± 1.0	91.7 ± 1.3	20.4 ± 37.2
DeepProbLog	90.8 ± 1.3	85.6 ± 1.1	timeout	timeout
DeepStochLog	90.8 ± 1.0	86.3 ± 1.9	92.1 ± 1.4	94.8 ± 0.9

must be able to use both the features of the documents and the symbolic network. Two well-known datasets for this task are the Cora⁴ (2708 nodes and 5429 edges) and Citeseer (3327 nodes and 9228 edges) [35].

T6: Word Algebra Problems. In this task, a natural language text describes a word algebra problem (e.g., *"Mark has 6 apples. He eats 2 and divides the remaining among his 2 friends. How many apples did each friend get?"*). This dataset of this task contains 300 training instances and was introduced in [32]. Each text contains 3 numbers, and all numbers have to be used exactly once in a formula containing addition subtraction, multiplication and division. The task is to predict the right numerical answer to the expression implied by textual description.

6.3 Results

For all experiments and metrics, we report the mean accuracy (or the mean most likely parse accuracy where applicable, i.e. **T1**, **T2** and **T3**) and standard deviation over 5 runs. We report "timeout" if a single of these 5 runs took more than 1 hour to execute.

Q1: Performance of DeepStochLog We first investigate whether DeepStochLog achieves state-of-the-art results compared to similar neural-symbolic frameworks. Table 1 shows the result for the MNIST addition task (**T1**), for training and testing on lengths 1 to 4. It shows that DeepStochLog performs similarly to the DeepProbLog [23] and NeurASP [43] frameworks but scales to larger sequences. Table 2 shows the result on the HWF task (**T2**). DeepStochLog performs similar to NGS and DeepProbLog for expressions of length 1 and 3. Starting from expression length 5, it becomes infeasible to train DeepProbLog. NGS [21] can still be trained, but for expression length 7, some runs fail to converge. DeepStochLog, however, performs well for all expression lengths. Table 3 shows the accuracy on task **T3**. Here we can see that both DeepStochLog and DeepProbLog achieve high accuracy, but DeepStochLog reaches a slightly higher accuracy for a longer length. For task **T6**, DeepStochLog and DeepProbLog achieve a similar accuracy of 94.8 ± 1.1 and 94.2 ± 1.4 respectively. We also compare to $\delta 4$ [3], but the authors only report the maximum accuracy reached. For all three frameworks, the maximum accuracy reached is 96.0. To conclude, DeepStochLog is able to achieve similar or better performance compared to other state-of-the-art neural-symbolic frameworks.

Q2: DeepStochLog scalability We now investigate whether DeepStochLog is more scalable than similar neural-symbolic frameworks. First, we observe that in the tasks **T1**, **T2**, **T4** and **T5**, DeepStochLog scales to settings or datasets that are infeasible for the competitors. Next, in Table 7, we compare the execution times for inference in task **T1**. We selected 100 queries from the training

⁴On the Cora dataset, another common task is link prediction, which is a purely symbolic task (i.e. there are no subsymbolic features on the documents).

Table 3: The parse accuracy (%) on the well-formed parentheses dataset (**T3**).

Method	Maximum expression length		
	10	14	18
DeepProbLog	100.0 $\pm$ 0.0	99.4 $\pm$ 0.5	99.2 $\pm$ 0.8
DeepStochLog	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0

Table 4: The accuracy (%) on the $a^n b^n c^n$ dataset (**T4**).

Method	Expression length		
	3-12	3-15	3-18
DeepProbLog	99.8 $\pm$ 0.3	timeout	timeout
DeepStochLog	99.4 $\pm$ 0.5	99.2 $\pm$ 0.4	98.8 $\pm$ 0.2

data and we computed the average time required from the system to compute the probability of the query. We repeated the experiment for increasing number lengths. DeepStochLog shows a huge gap over the competitors, especially for large numbers. The advantage of DeepStochLog over the competitors is two-fold. Firstly, DeepStochLog is natively implemented on top of SLG resolution and tabling, which plays a fundamental role in compactly representing derivations and SLD-trees. We analyzed the impact of tabling in Table 6, where we show the comparison between SLD and SLG resolution in DeepStochLog. In particular, we compared the resolution time required to find all the possible answers for expressions of variable lengths (on task **T2**). Secondly, DeepStochLog is based on a random walk semantics which is computationally cheaper than the possible world semantics exploited by DeepProbLog and NeurASP.

Q3: Larger scale relational datasets The complexity of many of the previous experiments comes from the large number of derivations for a single goal, while the number of subsymbolic inputs (e.g. images) in a single relational example was quite limited. Here, we focus on task **T5**, i.e. semi-supervised classification in citation networks, where the complexity mainly comes from the large number of elements of the unique relational example, i.e. the citation network. This task is usually out of the scope of (neural) PLP approaches due to the fact that there is a unique large relational example and the possible world semantics is prohibitive in this scenario. We compare against the following baselines: label propagation (LP) [44], semi-supervised embedding (SemiEmb) [42], manifold regularization (ManiReg) [1], skip-gram based graph embeddings (DeepWalk) [29], ICA [22] and GCN [19]. All these baselines are specific to the semi-supervised classification task, while DeepStochLog is a much more general framework. We finally tried to compare with DeepProbLog, which, however, does not scale to the size of this problem due to the different probabilistic semantics. Results are reported in Table 5. DeepStochLog compares similarly or favorably to most of the other methods, even though it is the only one that has not been developed for the specific task. However, it still underperforms w.r.t. ICA and GCN. But these methods use extra knowledge as input to the classifier in the form of precomputed or learned features of the neighbors of a document, which is very useful for this task but not considered in the DeepStochLog experiment. Adding or learning relational features for input to the neural modules is, however, an interesting future direction.

Q4: General programs in DeepStochLog Even though DeepStochLog naturally represents grammars for parsing sequences, NDCGs with Prolog goals are a powerful formalism to express more complex relational problems and programs. Actually, both task **T5** and **T6** have been solved with programs that depart from the pure grammar formalism and are more like general logic programs. We provide the complete models in Appendix B. The main ingredients are (neural) empty production rules, sometimes referred to as non-consuming or ϵ -production rules. They allow to take probabilistic decisions, including also neural networks, without consuming any element of the sequence, as shown in Example 4. This also shows that DeepStochLog has the full power of stochastic logic programs.

Table 5: **Q3** Accuracy (%) of the classification on the test nodes on task **T5**

Method	Citeseer	Cora
ManiReg	60.1	59.5
SemiEmb	59.6	59.0
LP	45.3	68.0
DeepWalk	43.2	67.2
ICA	69.1	75.1
GCN	70.3	81.5
DeepProbLog	timeout	timeout
DeepStochLog	65.0	69.4

Table 6: **Q4** Parsing time in seconds (**T2**). Comparison of the DeepStochLog with and without tabling (SLD vs SLG resolution).

Lengths	# Answers	No Tabling	Tabling
1	10	0.067	0.060
3	95	0.081	0.096
5	1066	3.78	0.95
7	10386	30.42	10.95
9	68298	1494.23	132.26
11	416517	timeout	1996.09

Table 7: Inference times in milliseconds for DeepStochLog, DeepProbLog and NeurASP on task **T1** for variable number lengths.

Numbers Length	1	2	3	4
DeepStochLog	1.3 ± 0.9	2.3 ± 0.4	4.0 ± 0.4	5.7 ± 1.8
DeepProbLog	13.5 ± 3.0	36.0 ± 0.5	199.7 ± 14.0	timeout
NeurASP	9.2 ± 1.4	85.7 ± 22.6	158.2 ± 47.7	timeout

Example 4 (Empty productions). We show a variant of the MNIST Addition problem using empty productions.

$$\begin{aligned}
 nn(mnist, [X], [Y], [digit]) &:: number(X, Y) \rightarrow []. \\
 addition(X, Y, N) &\rightarrow number(X, N1), number(Y, N2), N \text{ is } N1 + N2.
 \end{aligned}$$

This grammar will always produce the empty sequence but through the Prolog unification mechanism and the probabilistic modeling, we can express complex stochastic logic programs that include calls to neural networks.

7 Related Works

DeepStochLog is an expressive neural symbolic framework whose distinguishing features are: 1) it is based on the expressive stochastic logic programming paradigm, which can express probabilistic programs (as in T5-T6) as well as probabilistic unification based grammars (T1-T4); 2) it can work with both symbolic and subsymbolic data such as images (as shown in T1-T4); and 3) its inference and learning mechanism is based on SLG-resolution that naturally supports tabling, a form of dynamic programming (as shown in Q2).

There are several strands of related research. First, DeepStochLog is a neural logic programming language in the spirit of DeepProbLog [23], NeurASP [43], the neural theorem prover [31] and lifted relational neural networks (LRNNs) [37]. The first two systems are based on a probabilistic possible world semantics, while DeepStochLog is based on stochastic grammars, which—as we have shown—scales much better (in part also due to the use of tabling). The latter two approaches focus on Datalog (which cannot deal with function symbols) and use the logic to construct the neural network in a kind of knowledge based model construction approach. Furthermore, they are neither probabilistic nor do they deal with subsymbolic inputs such as images. Another related system is Tensorlog [5], which is based on stochastic logic programming. While sharing their roots in SLPs, it is less expressive than DeepStochLog, as it considers only Datalog and predicates of arity 2. While Tensorlog’s implementation is fast thanks to being in terms of tensors, it has only been applied to symbolic data.

Second, DeepStochLog can be viewed as a neural-based grammar, similarly to Neural Gram-